# CSE Hackathon on Data Structures and Algorithms

Department of Computer Science and Engineering

CMRIT, Bangalore

Date: _14-OCT-2026

**Instructions:**

1. Students can work in teams (up to 4 persons per team). The team members must be as per the registration.
2. All problems are required to be attempted. Atleast 1 problem to be solved using **Java** and one in **Python**. However, the teams can program in parallel.
3. The duration of the hackathon is from 9am-3pm (6 hrs) including breaks as needed.
4. Analysis:
    a. Working solution with a deep-dive analysis covering all possible corner cases are expected for each problem.
    b. Comparative analysis is required wherever applicable.
    c. Proper justification/proof as to why your solution is best/optimal is required.
5. Visualization:
    a. Visualization is required for each problem. You may use visualization tools (but not limited to) as below:
        i. https://visualgo.net/en
        ii. https://pythontutor.com/
        iii. Optional libraries: matplotlib, networkx, pygame, tkinter, or any web-based (HTML/JS) canvas.

## Problem 1: Maze with Keys and Doors

You are given an N x N grid. Each cell is one of: '.' (open cell), '#' (wall), 'S' (start), 'E' (exit), a lowercase letter 'a'-'f' (a key), or an uppercase letter 'A'-'F' (a door). You may move up, down, left or right, one cell per step. Stepping on a key cell picks up that key automatically (keys are never used up). A door 'A'-'F' can be entered only if you already hold the matching key 'a'-'f'. Find the length of the shortest path from S to E.

**Input Format:**

- Standard input line: T (number of distinct key/door types, $1 \le T \le 6$)
- Option 1: Read the NxN grid from a CSV file.
- Option 2: Generate the NxN grid randomly to form a valid maze containing T keys and T doors, with at least one solvable path.

**Output Format:**

- Output 1: Single integer shortest path length, or -1 if not possible.
- Output 2: A CSV file with the shortest path marked with ‘x’s (keys picked up and doors opened along the path must still be identifiable).

**Visualization:**

- Display the grid with colours:
    - White = open cell, Black = wall, Green = start, Red = exit; each key/door pair has its own unique colour.
- Highlight the path taken.

- Animate key pick-ups (key disappears, and a key-inventory panel updates) and door openings (door turns from its colour to white with an unlock mark).

## Problem 2: Closest Pair of Points Comparative Analysis

Given N points on a 2D plane, implement two algorithms to find the closest pair of points: the Brute-Force method and the Divide-and-Conquer method. Output the minimum distance and the pair of points for each. Ensure the algorithms handle all corner cases (e.g., duplicate points, collinear points, points sharing the same x or y coordinate, N = 2, N < 2). Then, analyse performance for increasing input sizes and verify that both algorithms always agree on the minimum distance.

**Input Format:**

- Standard Input: (integer) N
- Generate the next N coordinates (x, y) randomly (integer or floating point).

**Output Format:**

- First line (Brute-Force): Minimum distance (6 decimal places), followed by the two coordinates of the closest pair.
- Second line (Divide-and-Conquer): Minimum distance (6 decimal places), followed by the two coordinates of the closest pair.
- Third line: Execution time of each algorithm (in ms).

**Visualization:**

- Plot all points on a plane.
- Highlight the closest pair found by each algorithm (different colours) and draw the line joining them.
- Show the dividing line and the strip region (of width 2d) used by the Divide-and-Conquer algorithm.
- Plot a running-time vs N graph for both algorithms.
- Animate the recursive splitting and merging steps (Optional).

## Problem 3: Budget Traveller with Coupons

You must travel from city 1 to city N using one-way flights. Each flight has a ticket price. You are allowed to make at most K stops (intermediate cities) on the way. You also hold C discount coupons; a coupon can be applied to a single flight to halve its price (rounded down), and at most one coupon can be used per flight. Find the minimum total cost to reach N, or -1 if impossible.

**Input Format:**

- Option 1:
    - First line: N M
    - Next line: K C (maximum stops, number of coupons)
    - Next M lines: u v p (one-way flight from u to v with price p)
- Option 2:
- First line: K C
    - Read the input graph from a CSV file.
- Option 3:
    - Generate a valid graph with user specified K and C.

**Output Format:**

- Output 1: minimum cost or -1

- Output 2: CSV file of the graph with the path marked with ‘x’s (also indicate the flights on which coupons were used).

**Visualization:**

- Draw graph with nodes as cities and directed, weighted edges as flights.
- Mark start = green, destination = red, intermediate stops = orange.
- Highlight chosen path.
- Animate the journey: show the running cost and stops used, and change the colour of a flight (e.g., to gold) when a coupon is applied, displaying original price → discounted price.

## Problem 4: Exam Timetable Scheduling (Graph Colouring Challenge)

A university has N courses. Two courses conflict if at least one student is enrolled in both, so they cannot be scheduled in the same time slot. In addition, at most R exams can be held in any single slot (limited number of halls). Assign every course to a time slot so that no two conflicting courses share a slot and no slot exceeds R exams, while minimizing the total number of time slots. Output the minimum number of slots found and the complete timetable. Implement at least a greedy heuristic (e.g., Welsh-Powell or DSatur) and an exact method (e.g., backtracking / branch-and-bound), and compare them.

**Input Format:**

- Option 1:
    - First line: N M R
    - Next M lines: u v (courses u and v have a conflict)
- Option 2:
- First line: N R
    - Generate a random conflict graph with a user specified edge density.

**Output Format:**

- Output 1: Minimum number of slots (for each algorithm implemented), along with whether the result is proven optimal.
- Output 2: Slot-wise list of courses (e.g., Slot 1: 2, 5, 9).
- Output 3: CSV file with the timetable (rows = slots, columns = exam positions/halls).

**Visualization:**

- Draw the conflict graph with nodes coloured according to their assigned slot.
- Show the final timetable as a grid, with the same colour scheme as the graph.
- Show a comparison chart of slots used and running time for each algorithm as N increases.
- Animate the order in which courses are assigned to slots (Optional).

**ALL THE BEST**